

Proyecto 2: Informe Final: Repositorio.

Walther Alejandro Barrantes Yan

C31037

Resumen

Este proyecto desarrolla un sistema capaz de reconocer, a partir de una fotografía, las vocales (A, E, I, O, U) del alfabeto dactilológico de LESCO, la Lengua de Señas Costarricense. Para ello se construyó un conjunto de datos propio de 500 imágenes y se entrenaron y compararon tres enfoques de redes neuronales: un perceptrón multicapa (MLP) como línea base, una red neuronal convolucional (CNN) entrenada desde cero y un modelo de *transfer learning* basado en MobileNetV2. El mejor resultado lo obtuvo el *transfer learning*, con una exactitud del 82.7 % sobre el conjunto de prueba, seguido de cerca por el MLP (81.3 %), mientras que la CNN desde cero rindió muy por debajo (29.3 %). El proyecto no solo reporta estos resultados, sino también el proceso completo que llevó a ellos, incluyendo los errores detectados y corregidos durante el desarrollo, que resultaron ser una de las partes más instructivas del trabajo.

1. Introducción

En Costa Rica, las personas sordas se comunican mediante LESCO. Parte de esta lengua es el **alfabeto dactilológico**: una configuración de la mano para cada letra, que se usa sobre todo para deletrear nombres propios o palabras que no tienen una seña asignada. Reconocer automáticamente estas señas es un primer paso hacia herramientas de accesibilidad que, a mayor escala, podrían traducir señas a texto y apoyar la comunicación con la comunidad sorda.

El objetivo de este proyecto es entrenar un modelo que, dada la foto de una mano formando una vocal, prediga de qué vocal se trata. Se eligió trabajar con las **vocales** porque son señas *estáticas* (la mano permanece quieta), lo que las hace adecuadas para un clasificador de imágenes fijas; las señas con movimiento quedan fuera del alcance, ya que requerirían modelos de video.

Este trabajo es la continuación natural del Proyecto 1 del curso, en el que se resolvió un problema de clasificación de imágenes con un método clásico de aprendizaje automático (una máquina de soporte vectorial, o SVM). Aquí se da el salto al *aprendizaje profundo* (redes neuronales), que en lugar de recibir la imagen “aplanada” en una lista de números, aprende directamente de la estructura espacial de la imagen.

Los objetivos concretos fueron: (i) construir un conjunto de datos propio de señas de vocales; (ii) entrenar y comparar tres arquitecturas de redes neuronales de complejidad creciente; y (iii) analizar de forma crítica los resultados, apoyándose en la literatura reciente del área.

2. Trabajos relacionados

El reconocimiento de gestos de la mano (*Hand Gesture Recognition*, HGR) es un área muy activa dentro de la visión por computadora. Para orientar las decisiones de diseño de este proyecto, se revisaron tres publicaciones recientes de tipo *survey* (revisiones del estado del arte), prestando especial atención a las secciones de retos y trabajo futuro.

Foteinos et al. [3] presentan una revisión exhaustiva del reconocimiento visual de gestos con aprendizaje profundo, organizando el campo en tareas estáticas, dinámicas aisladas y continuas, y recopilando los conjuntos de datos y las métricas más usadas. Linardakis et al. [4] se centran específicamente en la entrada visual (imágenes RGB, de profundidad y video) y enumeran como retos abiertos el reconocimiento robusto en entornos reales, el manejo de oclusiones, la generalización entre usuarios distintos y la eficiencia computacional. Por su parte, Fertl et al. [2] abordan el HGR en dispositivos de borde (*edge*), conectando el problema con el hardware, el consumo y el despliegue en tiempo real, una perspectiva muy cercana a la Ingeniería Eléctrica.

De la lectura de estas fuentes se desprende una coincidencia importante: los tres trabajos señalan, entre los principales retos, la **sensibilidad al fondo y a la iluminación**, la **generalización a usuarios no vistos** y la **necesidad de conjuntos de datos grandes y diversos**. Estas observaciones guiaron directamente las decisiones del proyecto, tanto en la forma de recolectar el conjunto de datos (variando fondos, luz y ángulos) como en la elección de *transfer learning* para compensar el tamaño limitado del conjunto.

3. Metodología

3.1. Conjunto de datos

El conjunto de datos se construyó desde cero para este proyecto. Se tomaron **500 fotografías** de una mano formando cada una de las cinco vocales de LESCO, es decir, **100 imágenes por vocal**. Durante la captura se varió de forma deliberada el fondo (paredes, escritorios, ropa), la iluminación (natural y artificial) y el ángulo y la distancia de la toma. Esta variedad es intencional: si todas las fotos de una misma vocal tuvieran el mismo fondo, el modelo podría aprender el fondo en vez de la forma de la mano. La Figura 1 muestra algunos ejemplos ya procesados.

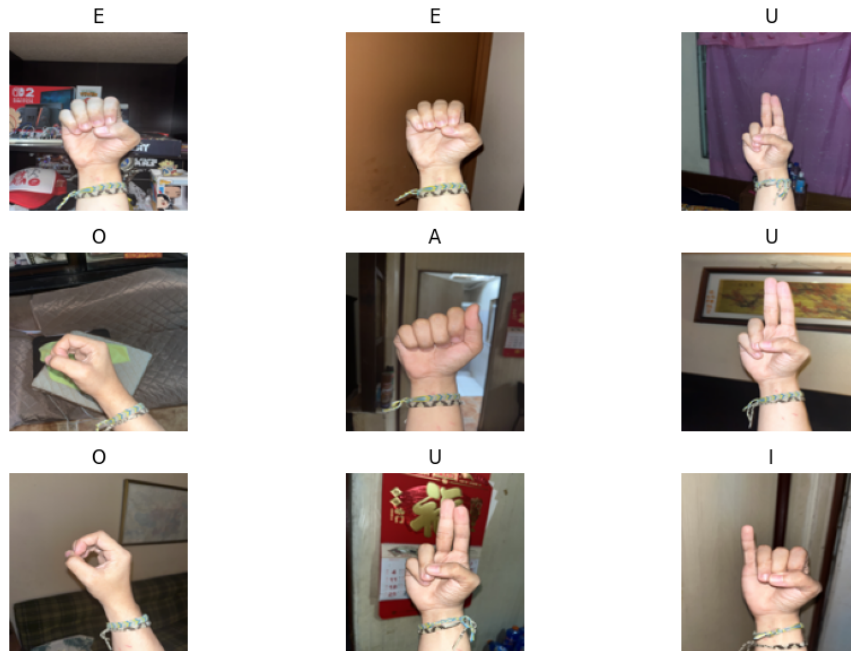


Figura 1: Ejemplos del conjunto de datos, redimensionados a 128×128 píxeles a color.

El preprocesamiento se automatizó en un cuaderno propio y consistió en: convertir las fotos a formato PNG (incluyendo el formato HEIC del iPhone), corregir la orientación a partir de los metadatos EXIF de la cámara, asegurar el modo de color RGB y redimensionar cada imagen a 128×128 píxeles. A diferencia del Proyecto 1, aquí las imágenes se mantienen **a color** y con su estructura bidimensional, ya que las redes convolucionales aprovechan precisamente esa estructura espacial. La clase de cada imagen se deduce automáticamente de la carpeta en la que se guarda.

3.2. Modelos

Se entrenaron tres modelos de complejidad creciente, todos con la biblioteca Keras/TensorFlow.

Modelo A — MLP (línea base). Un perceptrón multicapa que “aplana” la imagen y la procesa con capas densas. Es el equivalente en redes neuronales al enfoque del Proyecto 1 y sirve como punto de comparación. Al aplanar la imagen, pierde la información espacial, por lo que se espera que sea el más limitado para imágenes.

Modelo B — CNN (desde cero). Una red convolucional propia, formada por tres bloques de convolución con normalización por lotes (*Batch Normalization*) y activación ReLU, seguidos de una capa densa con regularización L2 y *Dropout*. Las convoluciones aprenden a detectar bordes y formas, aprovechando la estructura 2D de la imagen.

Modelo C — Transfer learning (MobileNetV2). En lugar de aprender desde cero, se reutiliza MobileNetV2 [5], una red ya entrenada sobre el gran conjunto de imágenes ImageNet [1]. Se “congela” esa base (se conservan sus pesos ya aprendidos) y solo se entrena una pequeña capa final adaptada a las cinco vocales. Es como partir de un modelo que ya sabe “ver” y enseñarle únicamente lo específico del problema.

A los modelos B y C se les aplicó *data augmentation* suave (pequeñas rotaciones y acercamientos), una técnica que genera variaciones artificiales de las fotos para que el modelo generalice mejor. Se evitó a propósito el volteo horizontal, porque voltear una seña de la mano cambiaría su significado.

3.3. Entrenamiento y evaluación

Las 500 imágenes se barajaron una sola vez y se dividieron en **70 % para entrenamiento (350 imágenes), 15 % para validación (75) y 15 % para prueba (75)**, cuidando que las cinco vocales estuvieran presentes en las tres particiones. La Tabla 1 muestra la distribución resultante. El conjunto de prueba nunca se usa durante el entrenamiento: sirve para medir qué tan bien generaliza el modelo a fotos nuevas.

Tabla 1: Distribución de imágenes por vocal en cada partición.

Partición	A	E	I	O	U	Total
Entrenamiento	69	69	67	66	79	350
Validación	19	19	14	16	7	75
Prueba	12	12	19	18	14	75

Para evaluar los modelos se usaron las métricas estándar de clasificación: **exactitud** (*accuracy*, porcentaje de aciertos), **precisión**, **exhaustividad** (*recall*) y **F1** (promedio macro), además de la **matriz de confusión**, que muestra en qué clases acierta y cuáles confunde. Todo el desarrollo se realizó en Google Colab con GPU, usando TensorFlow 2.20.

4. Resultados

La Tabla 2 resume el desempeño de los tres modelos sobre el conjunto de prueba. El mejor resultado lo obtuvo el *transfer learning* (82.7 % de exactitud), seguido muy de cerca por el MLP (81.3 %). La CNN entrenada desde cero quedó muy por debajo (29.3 %), apenas por encima del azar (que para cinco clases sería del 20 %).

Tabla 2: Comparación de los tres modelos en el conjunto de prueba (promedio macro).

Modelo	Exactitud	Precisión	Recall	F1	Parámetros
MLP (base)	0.813	0.841	0.821	0.819	12 616 709
CNN (desde cero)	0.293	0.367	0.357	0.279	4 288 997
Transfer (MobileNetV2)	0.827	0.826	0.816	0.817	2 264 389

La Figura 2 presenta la misma comparación de forma visual, incluyendo como referencia el SVM del Proyecto 1 (0.977). Conviene aclarar que ese valor corresponde a un **problema distinto** (clasificación de granos de arroz) y no es directamente comparable con este; se incluye únicamente como punto de contexto.

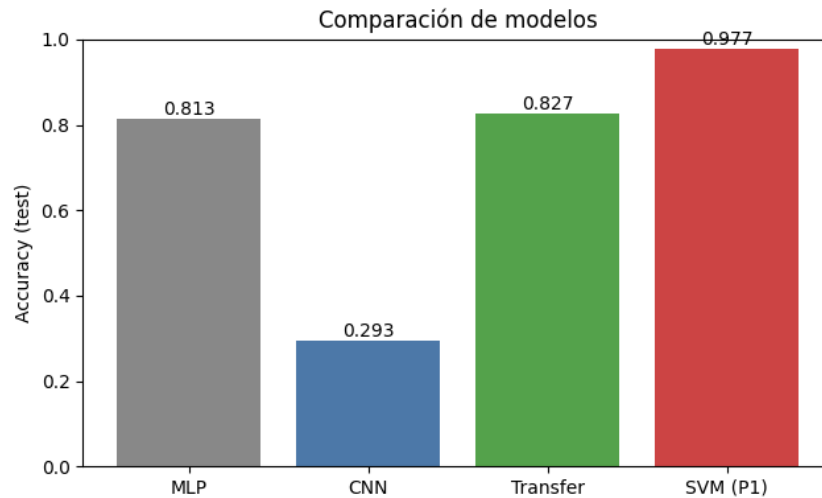
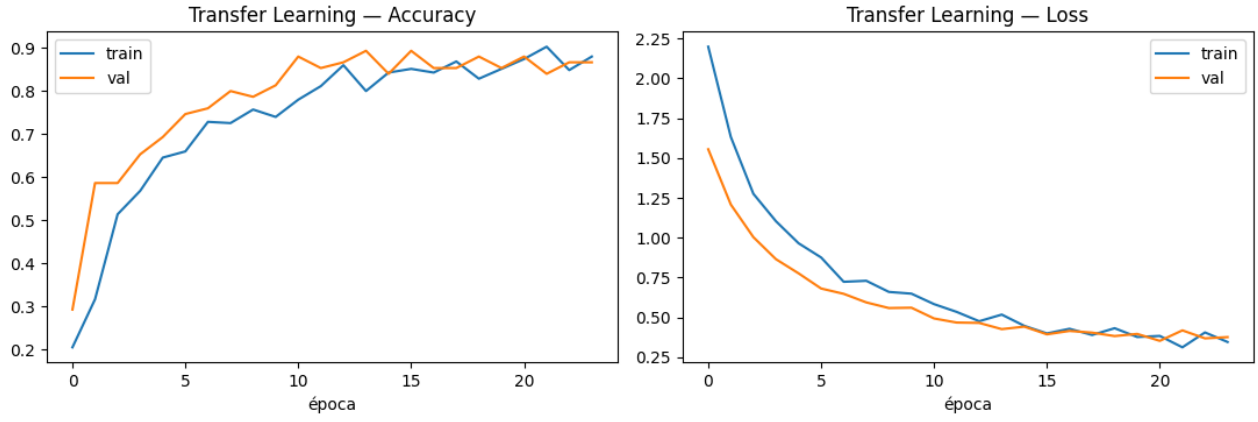
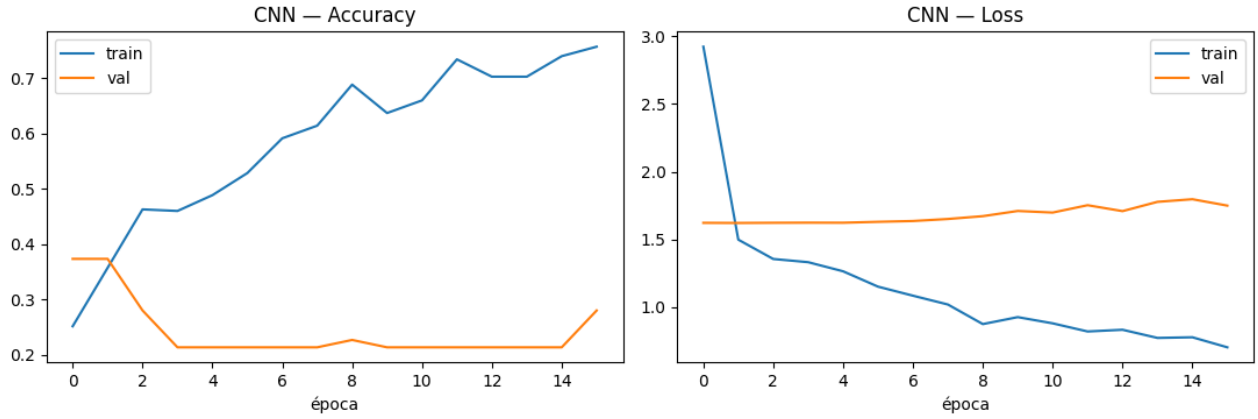


Figura 2: Exactitud de cada modelo en el conjunto de prueba.

Las curvas de entrenamiento (Figura 3) ayudan a entender el comportamiento de cada modelo. El *transfer learning* muestra un aprendizaje limpio y estable, con la validación acompañando al entrenamiento. El MLP aprende, pero su curva de validación es más inestable. La CNN, en cambio, evidencia el problema que se discute en la siguiente sección: la exactitud de entrenamiento sube mucho pero la de validación se queda estancada.



(a) Transfer learning (MobileNetV2).



(b) CNN desde cero.

Figura 3: Curvas de exactitud y pérdida durante el entrenamiento.

Finalmente, la Figura 4 muestra la matriz de confusión del mejor modelo. Se observa que el *transfer learning* acierta la mayoría de las vocales; las de mejor desempeño fueron la O y la U (F1 de 0.895 y 0.897), mientras que la mayor confusión se dio con la vocal E (recall de 0.667). El detalle por clase se resume en la Tabla 3.

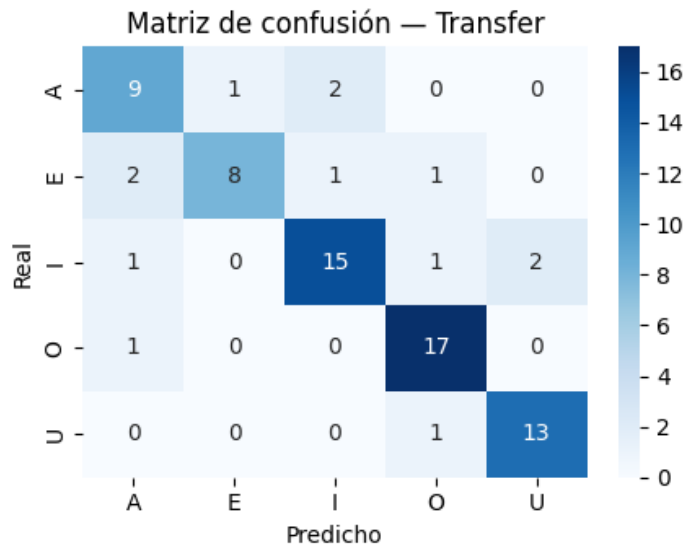


Figura 4: Matriz de confusión del modelo de *transfer learning* sobre el conjunto de prueba.

Tabla 3: Métricas por vocal del modelo de *transfer learning*.

Vocal	Precisión	Recall	F1	Soporte
A	0.692	0.750	0.720	12
E	0.889	0.667	0.762	12
I	0.833	0.789	0.811	19
O	0.850	0.944	0.895	18
U	0.867	0.929	0.897	14

5. Análisis y discusión

El orden de los resultados (transfer learning $\geq$ MLP $>$ CNN) tiene una explicación coherente y, en buena medida, esperada según la literatura revisada.

El **transfer learning** fue el mejor porque parte de un modelo (MobileNetV2) ya entrenado con millones de imágenes: sus primeras capas ya saben detectar bordes, texturas y formas, de modo que solo hace falta enseñarle la parte específica de las vocales. Esto le permite rendir bien incluso con un conjunto de datos pequeño.

El **MLP** obtuvo un resultado sorprendentemente alto para ser la línea base. Esto se explica por su gran cantidad de parámetros (más de 12 millones): con tan pocas imágenes, el MLP tiene capacidad de sobra para “memorizar” bastante bien el conjunto de entrenamiento, y el problema (cinco vocales sobre fondos variados) no es excesivamente difícil.

El caso más interesante es la **CNN entrenada desde cero**, que quedó muy por debajo. Esto no se debe a que la arquitectura sea mala, sino al **tamaño del conjunto de datos**. Entrenar una red convolucional desde cero requiere muchísimos ejemplos, y 500 imágenes son pocas. Este es, precisamente, uno de los retos que los tres *surveys* revisados señalan de manera recurrente: la dependencia de conjuntos de datos grandes y diversos [3, 4, 2]. En otras palabras, el bajo desempeño de la CNN no es un fracaso del proyecto, sino una confirmación práctica de una limitación bien documentada en el área.

6. Dificultades encontradas

Gran parte del aprendizaje de este proyecto provino de detectar y corregir problemas que no eran evidentes a primera vista. Se documentan aquí los tres más importantes.

6.1. Un error en la división de los datos

En las primeras corridas, las métricas no tenían sentido: un modelo mostraba una exactitud alta (cerca del 90 %) pero una F1 muy baja (cercana a 0.19), y los otros dos quedaban por debajo del azar. Esa combinación — exactitud alta con F1 baja — fue la pista clave: indicaba que el modelo estaba acertando casi siempre una sola clase sobre un conjunto de prueba dominado por esa clase.

La causa era la forma en que se dividían los datos. Al cargar las imágenes, estas venían ordenadas por clase (todas las A, luego todas las E, etc.), y la partición las cortaba en ese orden, sin mezclarlas. El resultado era que el entrenamiento se quedaba con unas vocales y la prueba con otras distintas, lo que hacía imposible acertar. La solución fue **barajar todas las imágenes una sola vez antes de dividir las**, de modo que cada partición contenga las cinco vocales de forma balanceada. Se añadió además una verificación automática que imprime cuántas imágenes de cada vocal caen en cada partición, para confirmar que el problema no se repita.

6.2. La inestabilidad de la CNN

La CNN fue el modelo que más costó estabilizar y pasó por varias etapas antes de llegar a su versión final:

- **Colapso a una sola clase.** En las primeras versiones, la red predecía siempre la misma vocal para todo. Se identificó que la causa principal era una tasa de aprendizaje (*learning rate*) demasiado alta, que hacía que la red “saltara” fuera de la zona útil. Bajarla de 0.001 a 0.0001 hizo que la red por fin empezara a aprender.

- **Sobreajuste (*overfitting*).** Una vez que aprendió, apareció el problema opuesto: la exactitud de entrenamiento subía hasta casi 0.98, pero la de validación se estancaba en torno a 0.40. Es decir, la red *memorizaba* el entrenamiento sin generalizar.
- **Regularización.** Para combatir el sobreajuste se probaron varias estrategias. Reducir la arquitectura con *Global Average Pooling* resultó contraproducente (la red volvía a colapsar), por lo que se optó por conservar la arquitectura que sí aprendía y añadirle regularización (*Dropout*, regularización L2 y *data augmentation* suave).

Aun con estos ajustes, la CNN no logró superar a los otros modelos, lo cual es consistente con la conclusión de la sección anterior: con tan pocos datos, hay un límite a lo que una red entrenada desde cero puede aprender. Lo valioso aquí no es el número final, sino el proceso de diagnóstico basado en evidencia (las curvas de entrenamiento y las matrices de confusión) que permitió entender *por qué* fallaba y *cómo* mejorarlo.

6.3. La inestabilidad del transfer learning

En algunas corridas, el modelo de *transfer learning* caía inesperadamente a un valor muy bajo. Al revisar sus curvas, se vio que el entrenamiento se detenía demasiado pronto (a las pocas épocas) y restauraba pesos de un modelo a medio entrenar. El criterio de *early stopping* (parada temprana) original vigilaba la pérdida de validación con poca paciencia y, ante un pequeño rebote, cortaba el entrenamiento antes de tiempo. Ajustar ese criterio para que vigilara la exactitud de validación con más paciencia devolvió al modelo a su desempeño real y estable ($\sim 82.7\%$).

7. Conclusiones

Este proyecto logró construir, de principio a fin, un sistema de reconocimiento de las vocales del alfabeto dactilológico de LESCO: desde la recolección de un conjunto de datos propio hasta un modelo capaz de clasificar fotografías nuevas. La comparación de tres enfoques permitió extraer conclusiones claras:

- El **transfer learning** con MobileNetV2 fue el mejor enfoque (82.7 % de exactitud), demostrando que reutilizar un modelo preentrenado es una estrategia muy efectiva cuando se dispone de pocos datos.
- El **MLP** funcionó bien como línea base (81.3 %), aunque por su naturaleza no aprovecha la estructura espacial de las imágenes.
- La **CNN entrenada desde cero** rindió por debajo de lo esperado (29.3 %), un resultado consistente con la literatura, que advierte sobre la necesidad de grandes cantidades de datos para este tipo de modelos.

Más allá de los números, el aprendizaje central del proyecto estuvo en el **proceso**: detectar que unas métricas no cuadraban, rastrear la causa hasta un error en la división de los datos, y diagnosticar la inestabilidad de la CNN a partir de sus curvas de entrenamiento. Ese trabajo de depuración, apoyado en la evidencia y en la literatura, fue tan formativo como el resultado final.

8. Trabajo futuro

Varias direcciones permitirían mejorar y ampliar este trabajo. La más evidente es **ampliar el conjunto de datos**, tanto en cantidad de imágenes como en diversidad de personas, para que el modelo generalice mejor a manos y condiciones distintas; esto beneficiaría especialmente a la CNN entrenada desde cero. También sería valioso **extender el vocabulario** a más letras o a números, y eventualmente a señas dinámicas, lo que requeriría modelos capaces de procesar video. Finalmente, desde la perspectiva de la Ingeniería Eléctrica, un paso natural sería el **despliegue en un dispositivo de borde** (por ejemplo, un sistema embebido), línea que la literatura sobre HGR en dispositivos *edge* identifica como especialmente relevante [2].

9. Repositorio

Todo el código, el conjunto de datos, el modelo entrenado y las figuras de este proyecto están disponibles en el repositorio de GitHub:

<https://github.com/Walther25/>

Referencias

- [1] Jia Deng et al. “ImageNet: A Large-Scale Hierarchical Image Database”. En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2009, págs. 248-255.
- [2] Elfi Fertl et al. “Hand Gesture Recognition on Edge Devices: Sensor Technologies, Algorithms, and Processing Hardware”. En: *Sensors* 25.6 (2025), pág. 1687. DOI: 10.3390/s25061687.
- [3] Konstantinos Foteinos et al. “Visual Hand Gesture Recognition with Deep Learning: A Comprehensive Review of Methods, Datasets, Challenges and Future Research Directions”. En: *arXiv preprint arXiv:2507.04465* (2025). DOI: 10.48550/arXiv.2507.04465.

- [4] Manousos Linardakis, Iraklis Varlamis y Georgios Th. Papadopoulos. “Survey on Hand Gesture Recognition from Visual Input”. En: *IEEE Access* (2025). DOI: 10.48550/arXiv.2501.11992.
- [5] Mark Sandler et al. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, págs. 4510-4520.